

BTS Services Informatiques aux Organisations (SIO)
Session 2025

Nom du candidat : Da Silva Peixoto

Prénom du candidat : Gabriel

Parcours : BTS SIO option SLAM

Fiche de réalisation professionnelle N° 6

Validation et les tests d'une API avec Postman

Période de réalisation : 03/03/26 au 04/02/26

Type de réalisation : En Stage, 2ème année.

Mode de réalisation : Individuelle

Sommaire

Contexte organisationnel	3
Problématique	3
Choix de la solution mise en œuvre	3
Résolution	3
Conclusions	4
Compétences	4

CONTEXTE ORGANISATIONNEL

Dans le cadre de ma formation en BTS SIO, j'ai initié et développé le projet "Asimov", un Système d'Information complet dédié à la gestion scolaire. Ce projet s'articule autour d'une architecture client-serveur robuste. J'ai été chargé de concevoir et de développer le Back-End sous la forme d'une API RESTful en Node.js et Express. Cette API constitue la véritable colonne vertébrale du projet, car elle a pour vocation d'alimenter simultanément plusieurs clients : une interface web dynamique et un client lourd développé en Java (JavaFX). L'API gère des données sensibles et une logique métier complexe, incluant la gestion des étudiants, des plannings, des notes, ainsi que l'authentification des différents acteurs (professeurs, administrateurs, élèves). Dans ce contexte multiprojets, la fiabilité du Back-End est un prérequis absolu avant tout développement côté Front-End.

PROBLÉMATIQUE

Le développement d'une API RESTful pose rapidement le défi de sa validation fonctionnelle et sécuritaire. Dans les premières phases du projet, je testais mes routes manuellement via mon navigateur ou des commandes terminal basiques. Cette approche s'est très vite révélée insuffisante, chronophage et source d'erreurs. Il était par exemple impossible de tester facilement les requêtes nécessitant des verbes HTTP spécifiques (POST, PUT, DELETE) ou d'injecter des données complexes au format JSON. Plus critique encore, la sécurisation des routes impliquait l'envoi de tokens d'authentification dans les en-têtes (Headers) HTTP, rendant les tests manuels ingérables. Sans une méthode de test structurée et automatisable, je prenais le risque de livrer une API instable, de générer des régressions lors de l'ajout de nouvelles fonctionnalités, et de ralentir considérablement le développement des interfaces clientes qui dépendaient de ces données.

CHOIX DE LA SOLUTION MISE EN ŒUVRE

Pour industrialiser la phase de test et garantir la fiabilité du service, j'ai choisi d'intégrer l'outil **Postman** à mon flux de travail. Plus qu'un simple client HTTP, Postman est une plateforme complète de développement d'API. Mon choix s'est porté sur cet outil pour trois raisons majeures. Premièrement, il permet de structurer les requêtes sous forme de "Collections", offrant ainsi une organisation logique par ressource (ex: utilisateurs, cours, notes).

Deuxièmement, il gère les variables d'environnement, ce qui est indispensable pour basculer facilement d'un environnement de développement local (localhost) à un serveur de production sans réécrire les requêtes, tout en stockant dynamiquement les tokens d'authentification. Enfin, Postman intègre un moteur d'exécution JavaScript (Postman Sandbox) permettant d'écrire de véritables scripts de tests automatisés pour valider les codes de statut HTTP, les temps de réponse et la structure des données JSON retournées.

RÉSOLUTION

La mise en œuvre de cette campagne de tests d'intégration s'est déroulée en plusieurs étapes méthodiques, transformant mon approche du développement Back-End.

Dans un premier temps, j'ai modélisé mon espace de travail Postman en créant une Collection exhaustive nommée "Asimov API". J'y ai répertorié l'ensemble des "endpoints", regroupés par dossiers thématiques. J'ai ensuite configuré mes environnements (Dev et Prod) en y définissant des variables globales telles que `{{base_url}}`. Ainsi, chaque requête était dynamique. Pour gérer la sécurité, j'ai configuré une requête de connexion (Login) qui, via un script écrit dans l'onglet "Tests" de Postman, récupérait automatiquement le token JWT renvoyé par le serveur pour le stocker dans une variable `{{auth_token}}`. Ce token était ensuite automatiquement injecté dans les en-têtes d'autorisation (Bearer Token) de toutes les requêtes subséquentes nécessitant des privilèges, m'évitant ainsi de fastidieux copier-coller.

Dans un second temps, j'ai rédigé des tests automatisés en JavaScript pour chaque route. Pour une requête de création d'utilisateur (POST), par exemple, le script vérifiait que le serveur renvoyait bien un code HTTP 201 (Created), que le temps de réponse était acceptable, et que le JSON de réponse correspondait strictement au schéma de données attendu. J'ai également scénarisé des tests d'échec pour vérifier la robustesse de l'API : tenter d'accéder à une route d'administration avec un compte étudiant pour m'assurer que le serveur renvoyait bien un code 403 (Forbidden). Enfin, j'ai utilisé l'outil "Collection Runner" de Postman pour exécuter l'intégralité de ces tests en un seul clic. Cela m'a permis de réaliser des tests de non-régression massifs à chaque modification majeure du code source de mon API.

CONCLUSIONS

L'intégration de Postman dans mon cycle de développement a radicalement amélioré la qualité et la stabilité du projet Asimov. La création de cette suite de tests d'intégration automatisés a éliminé le risque de régressions silencieuses et a garanti que la logique métier et les règles de sécurité étaient respectées de bout en bout. De plus, cette collection Postman, une fois exportée, a fait office de documentation technique vivante et interactive. Cela a considérablement facilité le développement ultérieur du client lourd JavaFX, puisque le format exact des requêtes à envoyer et des réponses JSON à traiter (Mapping JSON vers Objets Java) était déjà clairement établi, testé et documenté.

COMPÉTENCES

B1.5.2 Réaliser les tests d'intégration et d'acceptation d'un service : Conception, scénarisation et exécution automatisée d'une campagne de tests complète sur l'API RESTful via Postman pour en valider la fiabilité.

B1.1.1 Mettre en place et vérifier les niveaux d'habilitation associés à un service : Vérification rigoureuse des règles de sécurité des routes (vérification des tokens JWT, codes HTTP 401 et 403) pour s'assurer que seuls les utilisateurs autorisés accèdent aux données sensibles.

B1.1.7 Vérifier le respect des règles d'utilisation des ressources numériques : Audit des requêtes entrantes pour confirmer que les données soumises respectaient les formats attendus par le système (validation des schémas JSON).